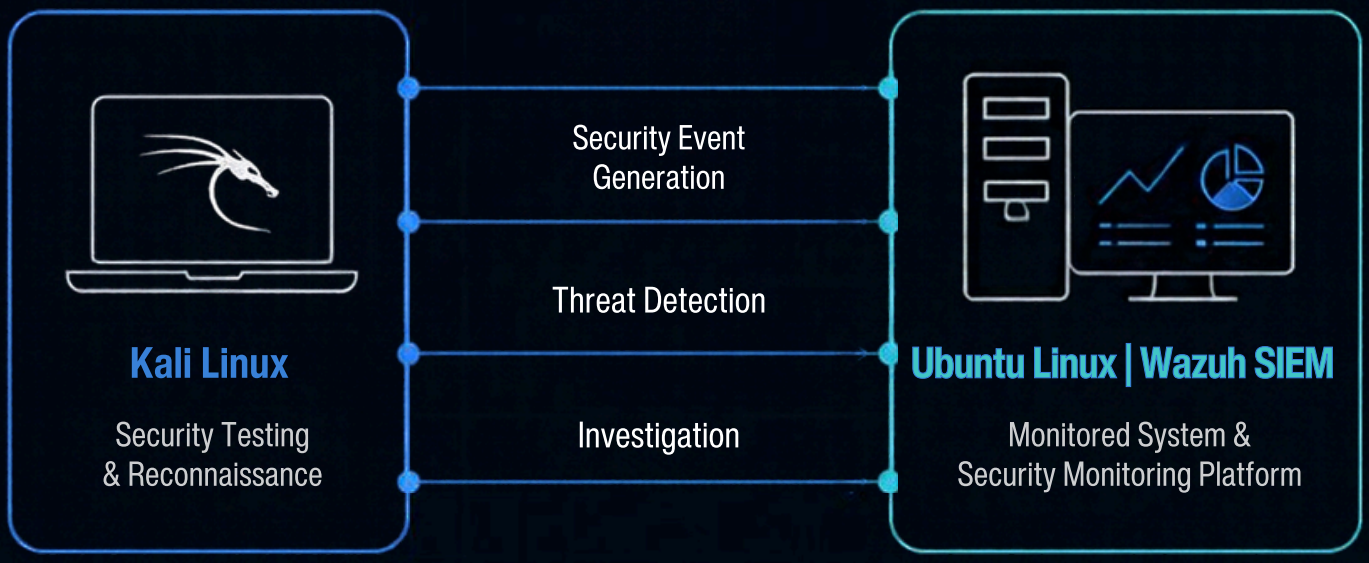
SIEM / SOC MINI LAB

Security Monitoring, Threat Detection & Incident Investigation



A hands-on lab demonstrating security event collection, detection engineering, and investigation using Wazuh SIEM.

Prepared by: Muhammad Umar Rizwan

Date: **18** October 2026

Project Details

Field	Details
Report title	Mini-SOC / SIEM Lab: Detection and Investigation Report
Author	Muhammad Umar Rizwan
Programme	BSc Cyber Security, Curtin University Dubai
Date	18 October 2026
Environment	Two VirtualBox VMs: Kali Linux, and Ubuntu Desktop running Wazuh and Suricata
Purpose	Personal cybersecurity portfolio project

Summary

This project is a small SIEM/SOC lab that I built to practise the kind of work an entry-level blue team analyst does: seeing security events, finding out what caused an alert, and turning a repeated pattern into a detection rule. I built it in VirtualBox with two virtual machines. Kali Linux was used to create the test activity. Ubuntu Desktop ran an all in one Wazuh setup and was also the machine being watched. Later I added Suricata so I could see network traffic as well as login events.

I started with SSH login monitoring. A failed login with the made-up username “labinvaliduser” showed up in the Ubuntu logs and in Wazuh under built-in rule 5710 (level 5). Then I used Hydra from Kali to run a small, limited brute-force test, which triggered built-in rule 5712 (level 10). At this stage I was learning how the existing detections and event fields work, not writing new rules.

For the network test, I ran Nmap from Kali against the Ubuntu host-only address, 192.168.56.108. Suricata raised alerts for the TCP SYN probes and Wazuh collected them. I then wrote custom Wazuh rule 100200 to match the Suricata alert, and rule 100201 to catch repeated alerts from the same source IP in a short time. The level 10 alert is called “possible port scan” because the rule does not prove that six different ports were hit. It only shows that the same source set off six probe alerts quickly. I also ran into a few problems along the way, which I describe in the Troubleshooting section.

1. Introduction

1.1 Why I built the lab

Reading about SIEM tools is useful, but it does not show how an alert is really created. An analyst needs to know where an event comes from, whether it reached the monitoring tool, which rule matched it, and what the evidence really shows. I wanted to practise all of that in a small lab that I could control and fix myself.

1.2 Scope

The lab covers virtual networking, installing Wazuh and opening the dashboard, agent visibility, SSH login events, a repeated failed-login test using Hydra, Nmap scanning, Suricata network alerts, and custom Wazuh rules. This report shows what I built and what I saw. It does not claim to cover every kind of attack.

I did not test any public systems or anyone else's network, and I did not try any destructive attacks. This is not a production setup or a formal penetration test.

2. Objectives and Success Criteria

2.1 Objectives

- Install Wazuh and open the dashboard from the Windows host.
- Check that Kali shows up as a Wazuh agent and look at the events created on Ubuntu.
- Create SSH login failures and find the matching events in Wazuh.
- Use Hydra for a small brute-force test and check that the built-in higher-severity SSH rule fires.
- Use Nmap to send a small set of TCP SYN probes at the Ubuntu lab host.
- Get Suricata alerts into Wazuh and write custom rules for single probes and for repeated probes from one source.
- Keep evidence organised and write down the lab's limits.

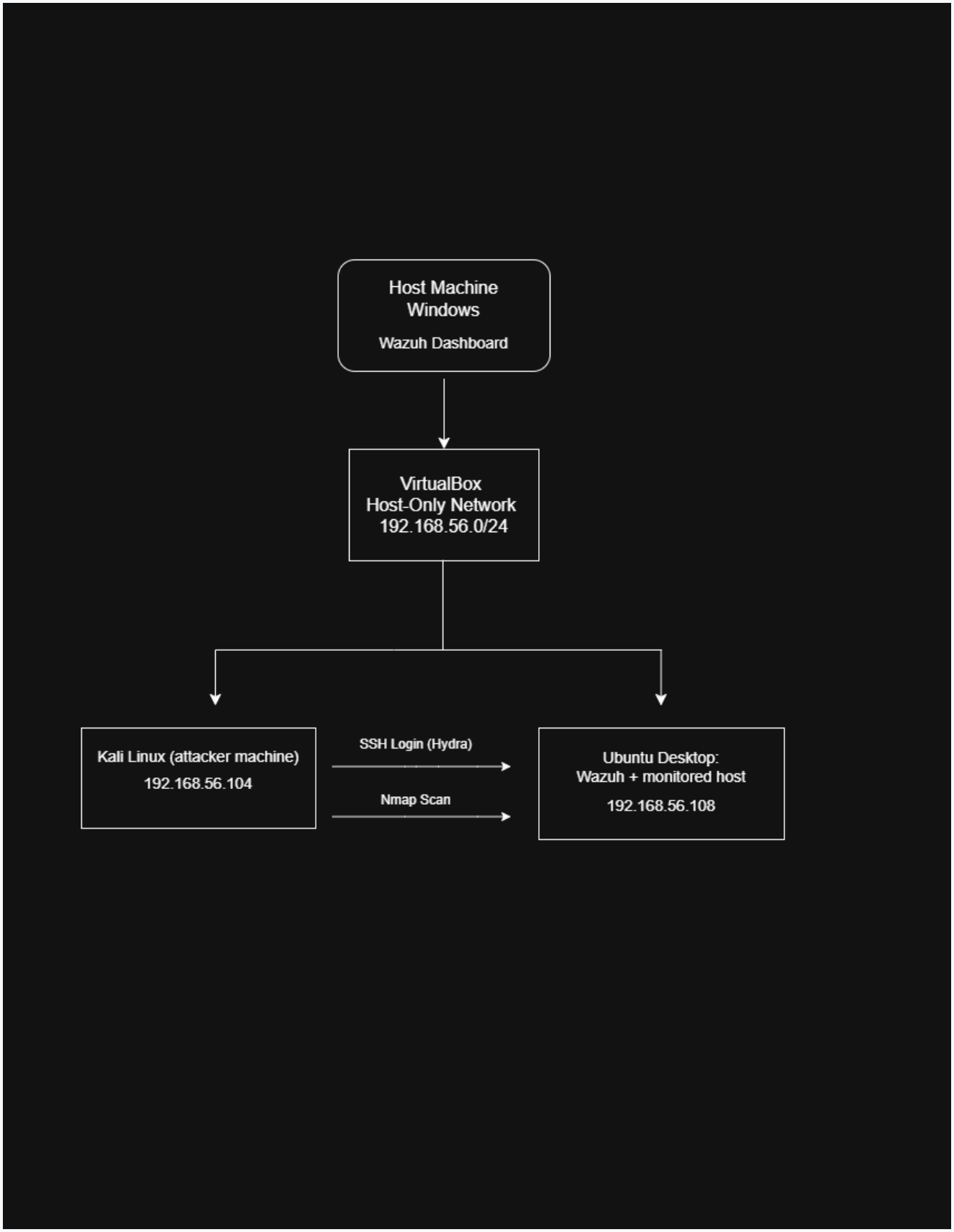
2.2 Completion summary

Objective	Result	Evidence reference
Dashboard access	Achieved	Figure 3
Kali agent status	Achieved	Figure 4
SSH failed-login monitoring	Achieved	Figures 6 and 7
Repeated SSH failures (Hydra)	Achieved (built-in rule 5712)	Figures 8 and 9
Suricata telemetry in Wazuh	Achieved	Figure 10
Custom probe rule 100200	Achieved	Figure 13
Custom correlation rule 100201	Achieved (level 10 alert seen)	Figures 14 and 15

3. Architecture and Lab Environment

3.1 High-level design

The lab uses two VirtualBox VMs. Each one has a host-only adapter for talking to the other VM and a NAT adapter for internet access. The host-only network carried the Kali-to-Ubuntu tests and let me open the Wazuh dashboard from Windows. I kept my existing VirtualBox host-only network instead of making a new one, so I would not break the network setup.



Architecture diagram: Windows host, VirtualBox host-only network, Kali Linux, and Ubuntu/Wazuh.

Figure 1. Architecture diagram: Windows host, VirtualBox host-only network, Kali Linux, Ubuntu/Wazuh and Suricata.

3.2 Components and addresses

Component	Role / configuration
Windows host	Runs VirtualBox and accesses the Wazuh dashboard.
Kali Linux VM	Source of the test traffic; host-only address 192.168.56.104.
Ubuntu Desktop VM	Ubuntu 26.04.1 Desktop. Monitored target and all-in-one Wazuh host; host-only address 192.168.56.108 on enp0s3.
Ubuntu NAT interface	10.0.3.15 on enp0s8, used for internet access.
Wazuh	Version 4.14.8; all-in-one manager, indexer, Filebeat and dashboard.
Suricata	Version 8.0.3; capture interface enp0s3; EVE JSON at /var/log/suricata/eve.json.
Dashboard URL	https://192.168.56.108
Kali agent	Name Kali-Linux, agent ID 001, status Active.

3.3 Why the design is simplified

Ubuntu has two jobs: it runs Wazuh and it is also the machine being watched. This meant fewer VMs and made the project easier to run on a personal computer. The downside is that the SIEM and the target are not separate. If the Ubuntu VM stops working, I lose both the monitored service and the monitoring. So this is a simple learning setup, not a production design.

4. Tools Used

These are the tools I used, grouped by what they do.

4.1 Main platforms

Tool	What I used it for
Oracle VirtualBox	Made and managed the Kali Linux and Ubuntu VMs, with NAT and host-only networking.
Kali Linux	Testing machine. Used to create SSH login failures, run the brute-force test, run Nmap scans and generate network traffic.
Ubuntu Linux	Monitored machine and SIEM server. Runs Wazuh all-in-one and OpenSSH, and provides the system and login logs.
Wazuh 4.14.8	The core of the lab. Used for endpoint monitoring, collecting events, alerts, threat hunting and custom detection rules.

4.2 Security testing and detection tools

Tool	What I used it for
Nmap	Network scanning, port scanning, and service and version detection.
Hydra	A small, controlled SSH brute-force test.
OpenSSH	SSH service on Ubuntu, which created login events.
tcpdump	Captured network packets, including scan-related SYN and RST traffic.
Suricata	Network threat detection, using local rules for Nmap-related traffic.
Wazuh agent	Sent endpoint security data to the Wazuh manager.

5. Methodology and Safety

5.1 Workflow

I followed the same steps each time: check the lab target, create a test event, find the logs it produced, open the matching alert, and write down what the evidence really shows. For the network test I looked at the packets too, as well as the Suricata and Wazuh events, instead of assuming Nmap was detected just because it printed results.

5.2 Safety controls

- All tests were aimed at my own Ubuntu VM on the host-only network.
- The Nmap scan used a list of six ports and the slow -T2 timing setting.
- For SSH testing I used a made-up lab username, and Hydra with a limited number of attempts.
- I took evidence from the lab dashboard and configuration files.
- Do not run the commands in this report on systems you do not have permission to test.

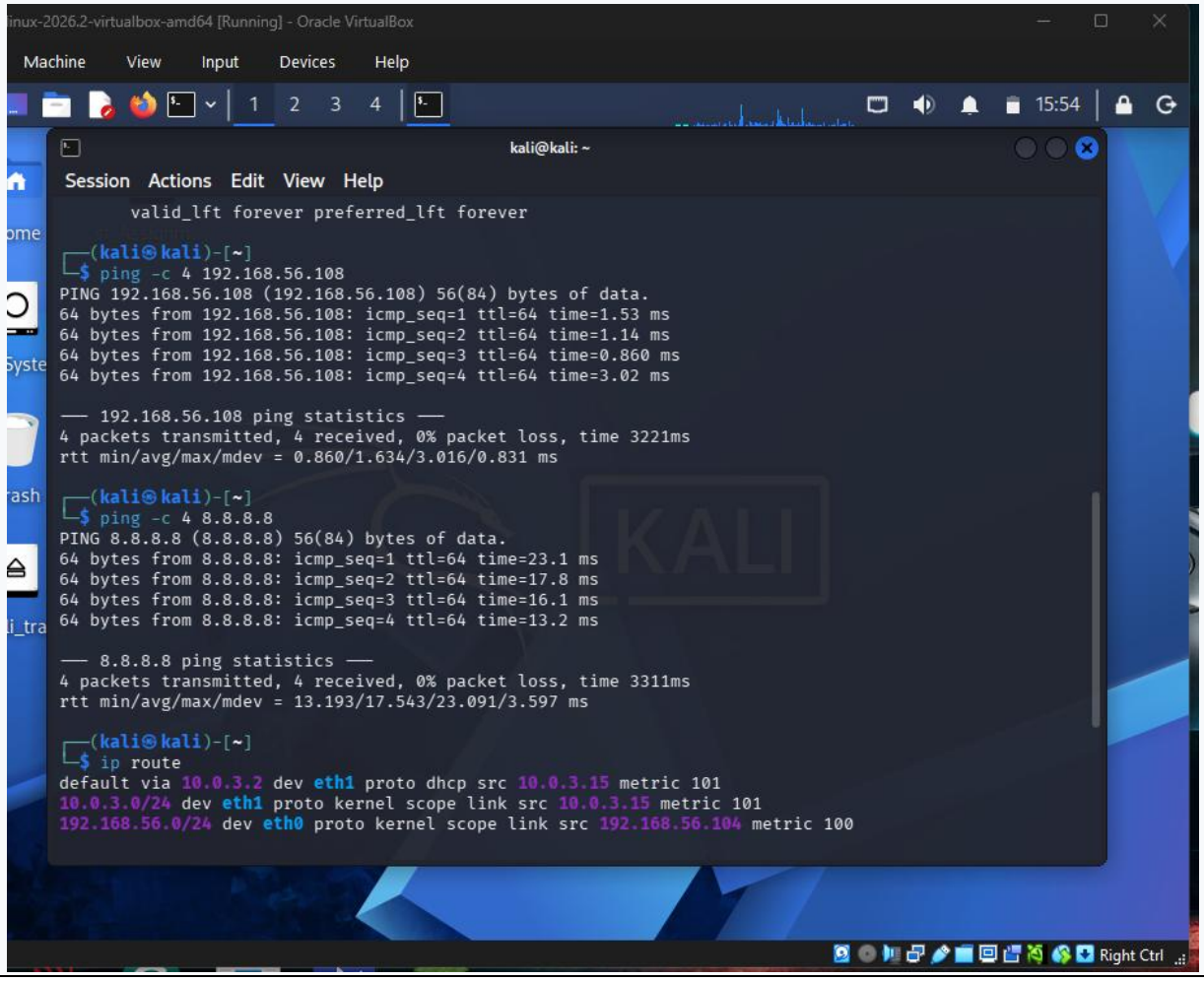
5.3 How I handled evidence

A screenshot should show enough to check the claim, such as the rule ID, alert level, source or user field and, if possible, the time. I used screenshots only as supporting evidence, so the steps and results are also explained in the text.

6. Implementation and Configuration

6.1 VirtualBox networking

Each VM had a host-only adapter and a NAT adapter. The host-only network was 192.168.56.0/24. The VirtualBox host-only adapter was 192.168.56.1, and the two VMs were 192.168.56.104 (Kali) and 192.168.56.108 (Ubuntu) during the tests. Kali could ping Ubuntu, and both VMs had internet during setup. What mattered was that both VMs were on the right network, on the same subnet, and that test traffic actually reached the Ubuntu target.


inux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
Machine View Input Devices Help
15:54
kali@kali: ~
Session Actions Edit View Help
valid_lft forever preferred_lft forever
(kali㉿kali)-[~]
$ ping -c 4 192.168.56.108
PING 192.168.56.108 (192.168.56.108) 56(84) bytes of data.
64 bytes from 192.168.56.108: icmp_seq=1 ttl=64 time=1.53 ms
64 bytes from 192.168.56.108: icmp_seq=2 ttl=64 time=1.14 ms
64 bytes from 192.168.56.108: icmp_seq=3 ttl=64 time=0.860 ms
64 bytes from 192.168.56.108: icmp_seq=4 ttl=64 time=3.02 ms
— 192.168.56.108 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3221ms
rtt min/avg/max/mdev = 0.860/1.634/3.016/0.831 ms
(kali㉿kali)-[~]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=23.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=17.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=16.1 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=13.2 ms
— 8.8.8.8 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3311ms
rtt min/avg/max/mdev = 13.193/17.543/23.091/3.597 ms
(kali㉿kali)-[~]
$ ip route
default via 10.0.3.2 dev eth1 proto dhcp src 10.0.3.15 metric 101
10.0.3.0/24 dev eth1 proto kernel scope link src 10.0.3.15 metric 101
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.104 metric 100
Right Ctrl

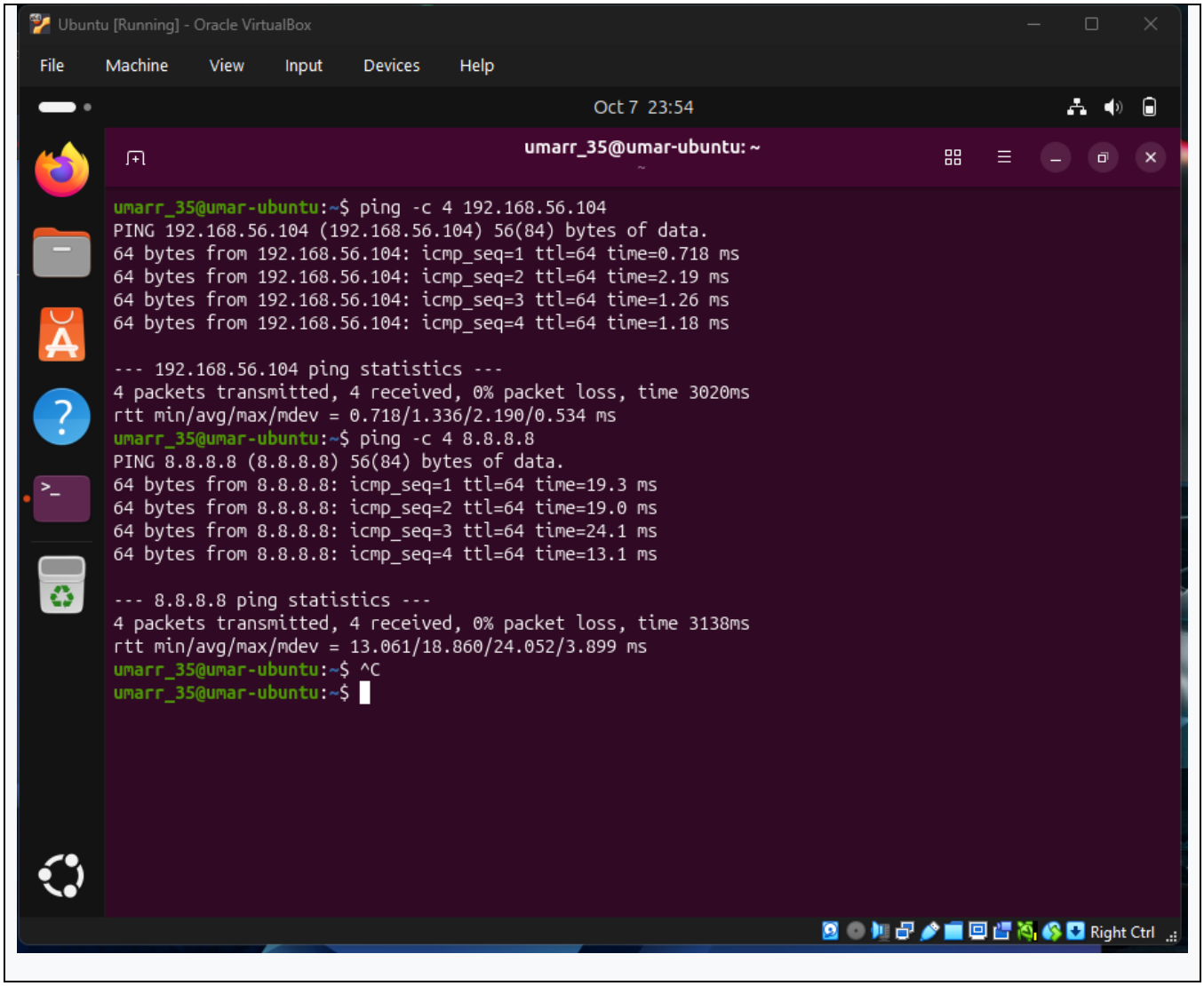
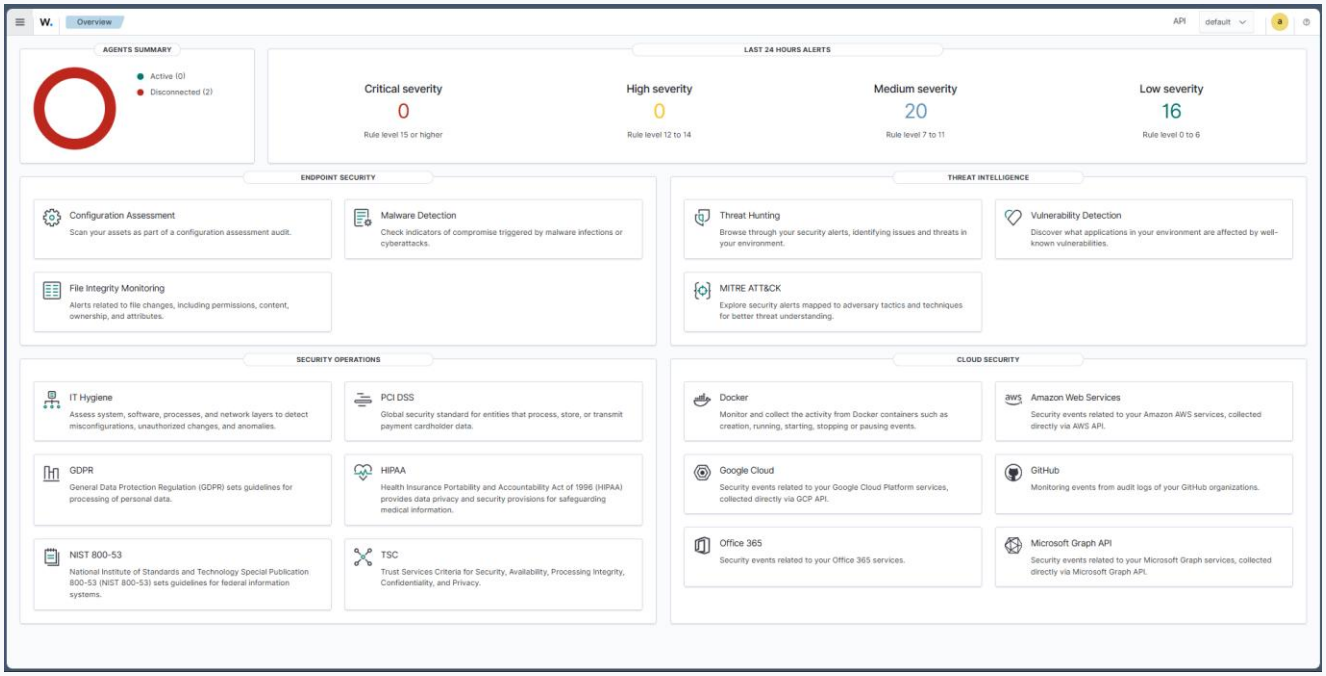


Figure 2. Connectivity check showing Kali can reach the Ubuntu host-only address.

6.2 Wazuh installation and service checks

I installed Wazuh 4.14.8 as an all-in-one setup on Ubuntu. After installation the manager, indexer, Filebeat and dashboard all showed as active, and I opened the dashboard from the Windows host at https://192.168.56.108. This became my main place for checking agent status, searching events and reading alert details.

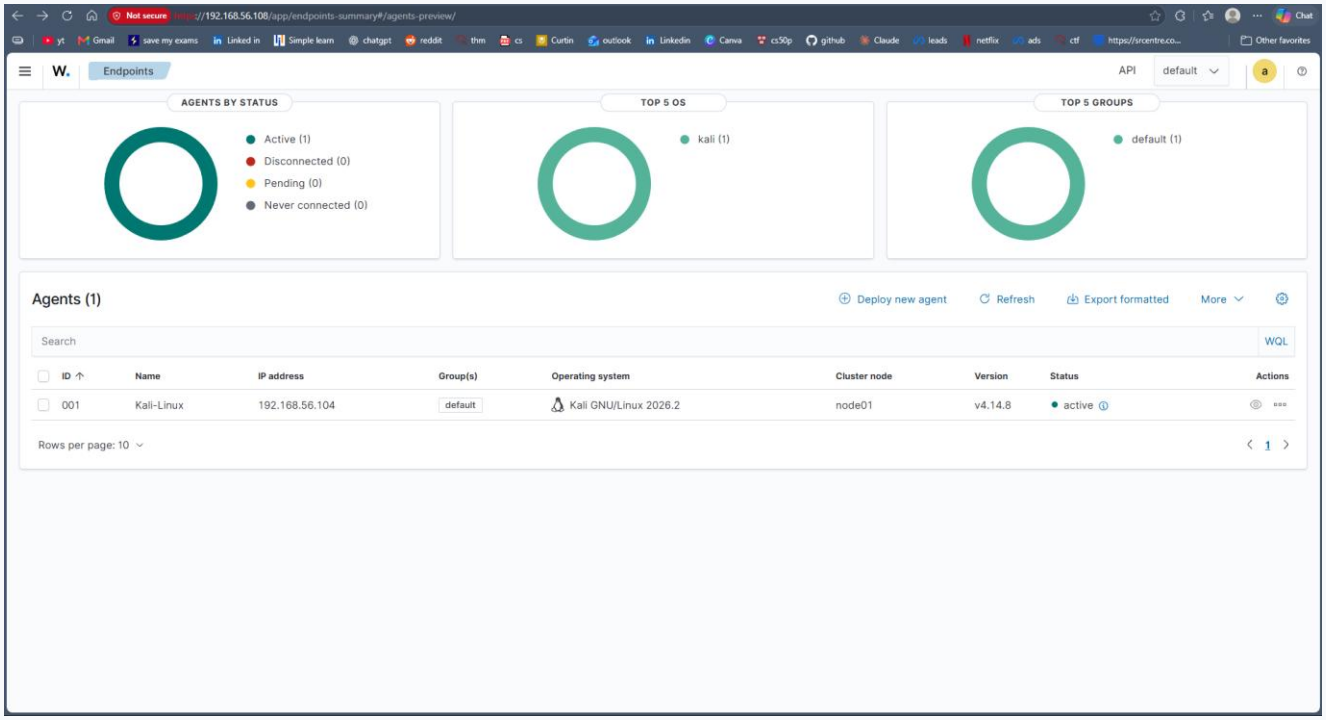


File: 01-wazuh-dashboard.png
Wazuh dashboard accessible from the Windows host.

Figure 3. Wazuh dashboard accessible from the Windows host.

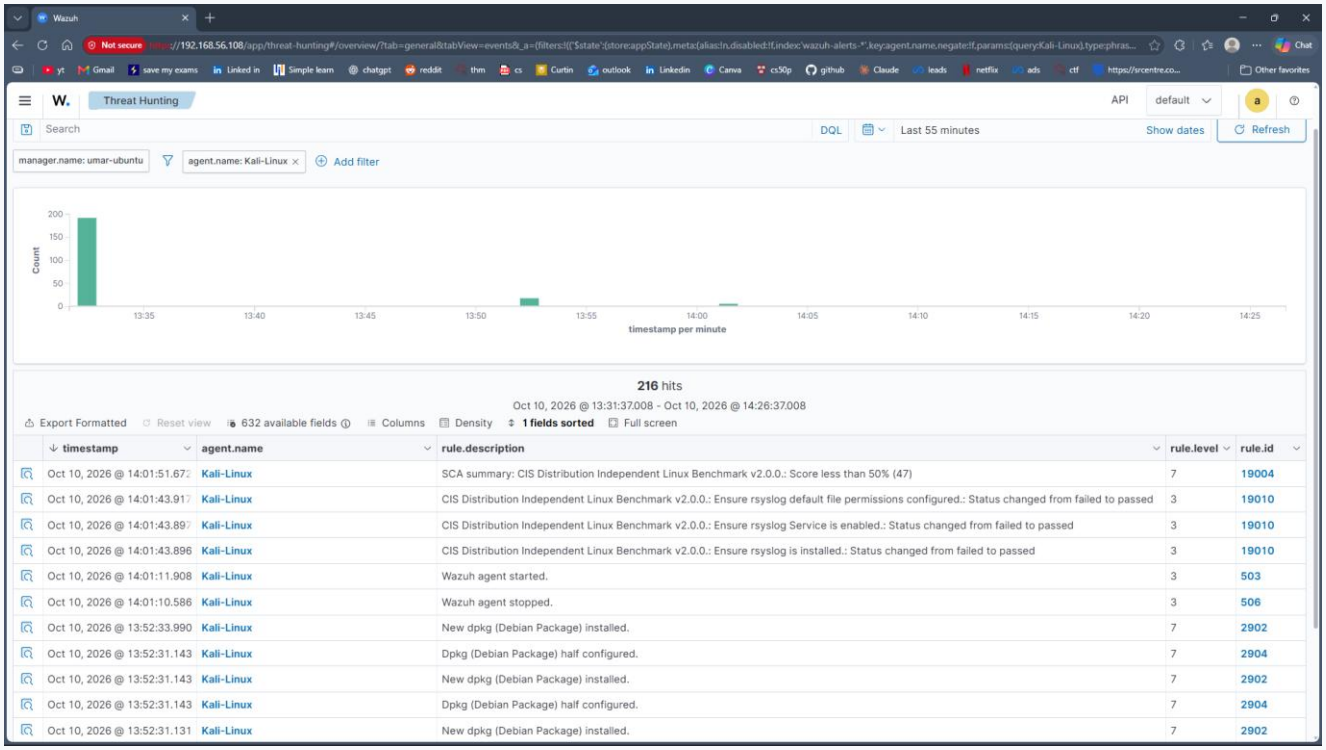
6.3 Agent visibility

In Wazuh, the Kali machine showed up as Kali-Linux, ID 001, with status Active. Events from Ubuntu itself also appeared in the dashboard under the local Ubuntu name.



File: 02-kali-agent-active.png
Wazuh Agents view showing Kali-Linux, ID 001, Active.

Figure 4. Wazuh Agents view showing Kali-Linux, ID 001, Active.



File: 03-kali-security-events.png
Threat Hunting / Events view showing security events from the lab.

Figure 5. Threat Hunting / Events view showing security events from the lab.

6.4 Suricata and Wazuh event collection

Suricata 8.0.3 ran on Ubuntu and watched traffic on enp0s3, the host-only interface. It wrote its alerts in EVE JSON format to /var/log/suricata/eve.json, and Wazuh collected that file. Suricata alerts then showed up in Wazuh through built-in rule 86601. This gave the lab network data as well as login logs.

7. Test Scenarios and Results

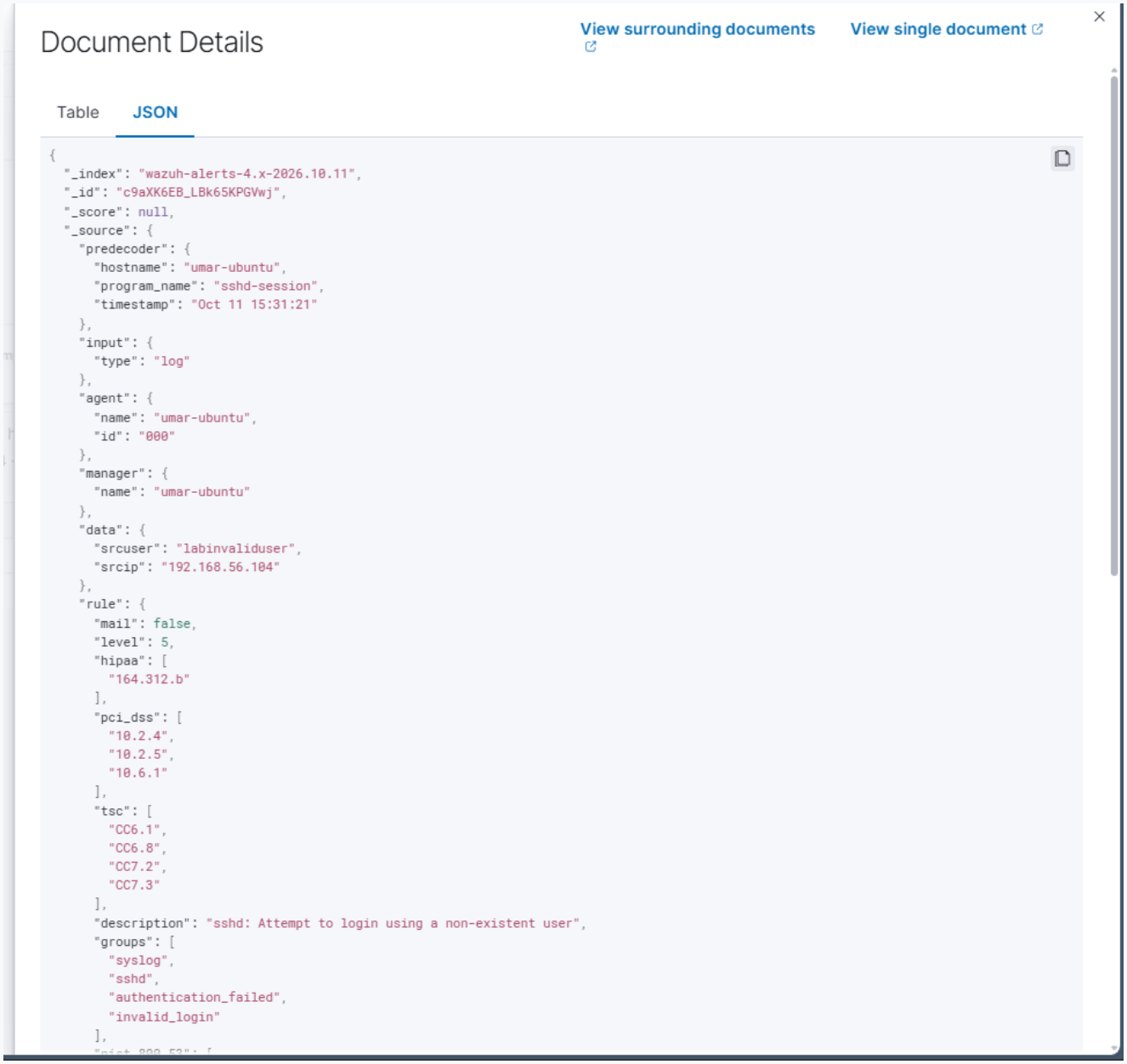
7.1 Test A – SSH failed-login detection

Objective. Check that a failed SSH login on the Ubuntu lab machine leaves logs that Wazuh can collect and search.

Procedure. From Kali I tried to log in over SSH with a username that does not exist, labinvaliduser. I checked the Ubuntu login logs first, then searched Wazuh for data.srcuser: labinvaliduser.

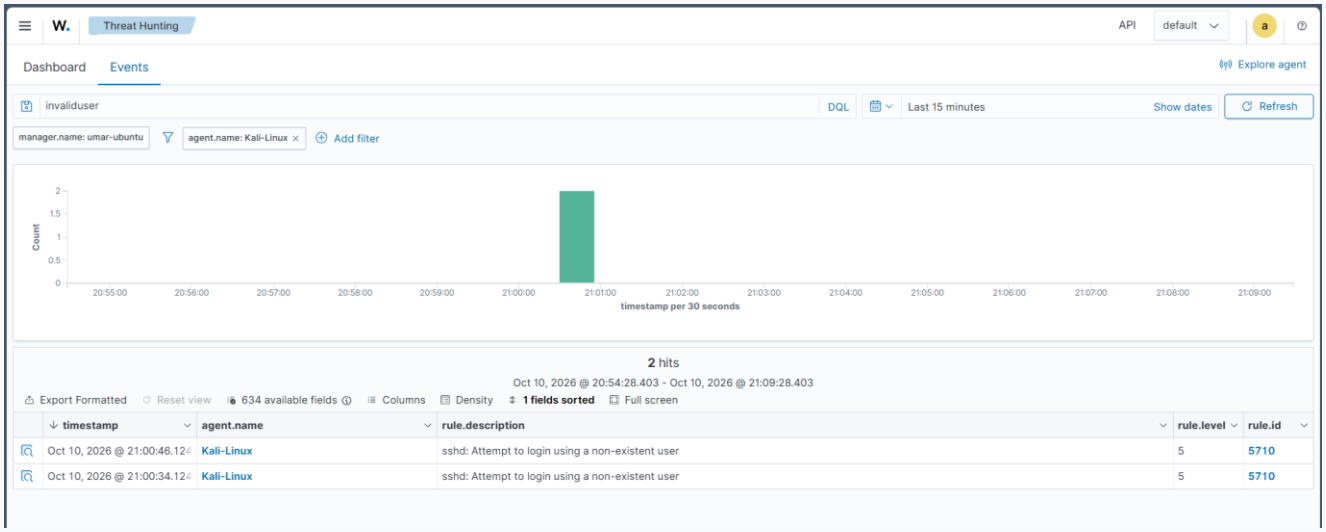
Observed result. The Ubuntu JSON/login logs showed the test username. Wazuh returned matching events for the Ubuntu machine, including built-in rule 5710 at level 5.

Interpretation. This shows that a failed login reached Wazuh. One failed login is not enough to call it brute force. It is just an event that may need a closer look, depending on the context and whether it repeats.



File: 08-wazuh-ssh-invaliduser-json.png
Ubuntu JSON evidence showing labinvaliduser.

Figure 6. Ubuntu JSON evidence showing labinvaliduser.



File: 05-wazuh-ssh-failed-login-detected.png

Figure 7. Wazuh showing the failed SSH login event.

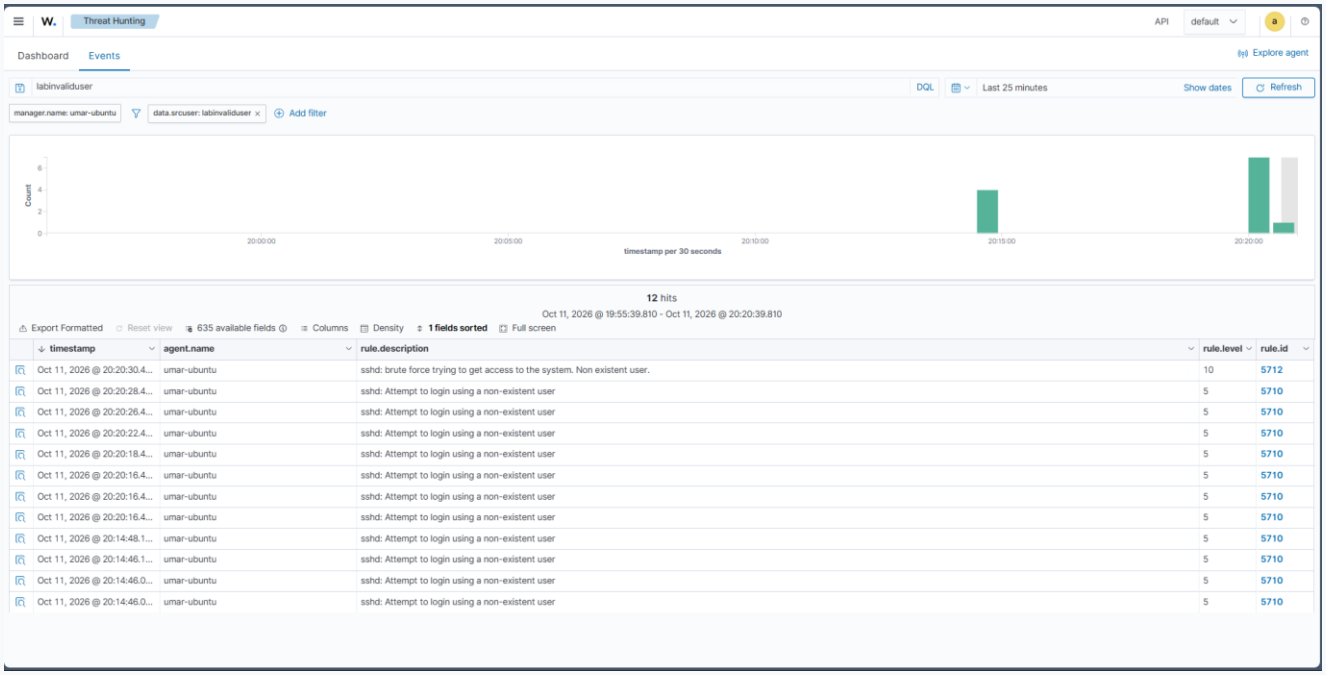
7.2 Test B – Repeated SSH failures with Hydra

Objective. Check a built-in higher-severity rule for repeated SSH login failures.

Procedure. I used Hydra from Kali to run a small, limited SSH brute-force test against the Ubuntu lab machine, using the made-up username labinvaliduser. Then I checked Wazuh Events for the higher-severity alert.

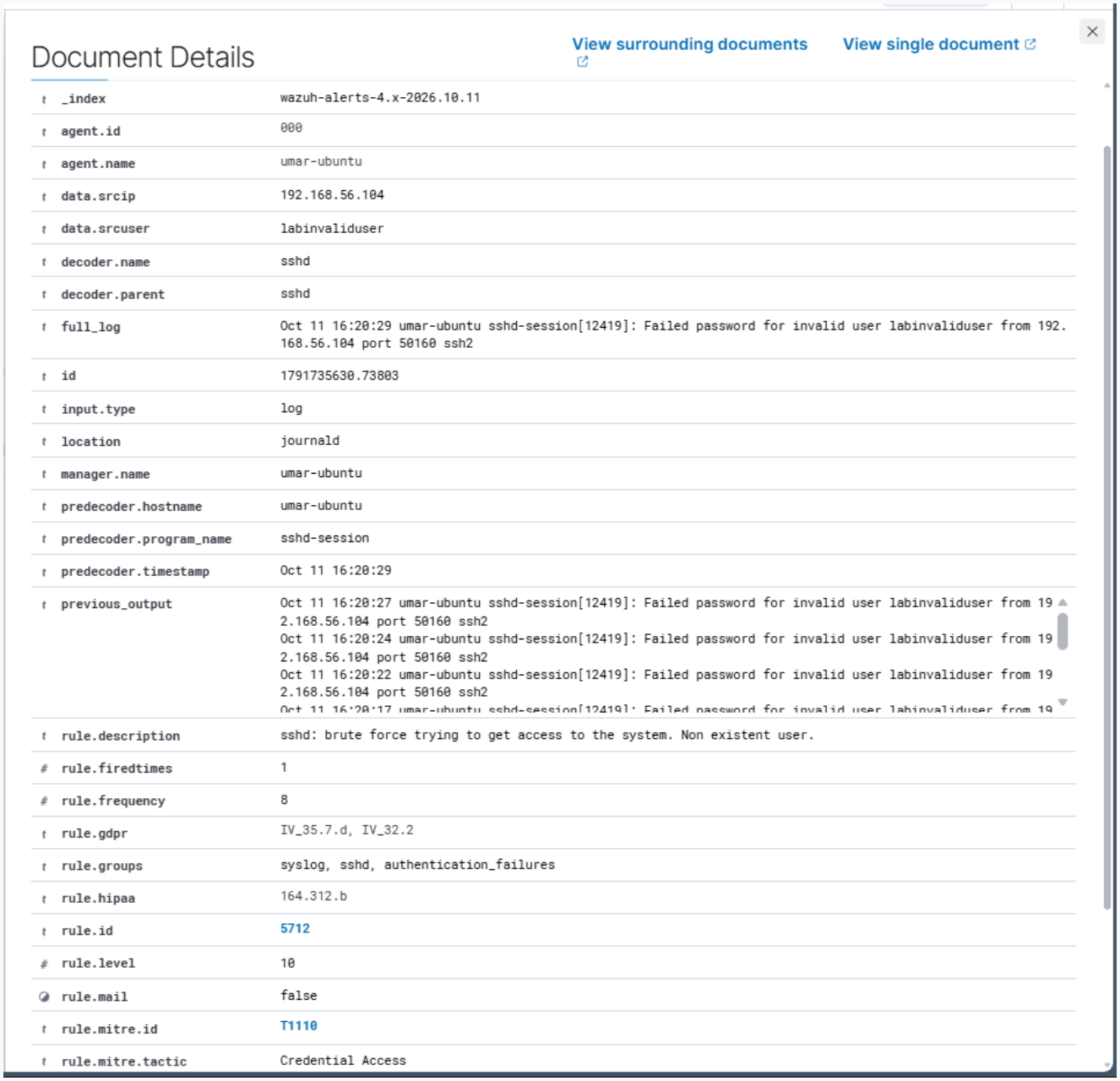
Observed result. After the repeated failed attempts, Wazuh showed built-in rule 5712 at level 10, with labinvaliduser involved. The alert details showed the Kali source address 192.168.56.104 and the Ubuntu machine.

Interpretation. This test checked a built-in brute-force rule. I did not write it. The alert is a useful warning, but it does not prove that anyone got in.



File: 09-wazuh-ssh-brute-force-alert.png
Built-in Wazuh Rule 5712 level 10 alert.

Figure 8. Built-in Wazuh Rule 5712 level 10 alert.



File: 10-wazuh-ssh-brute-force-alert-details.png
Expanded Rule 5712 details; check source IP, username and timestamp are visible.

Figure 9. Expanded details for rule 5712, including the source IP and username.

7.3 Test C – Nmap TCP SYN reconnaissance

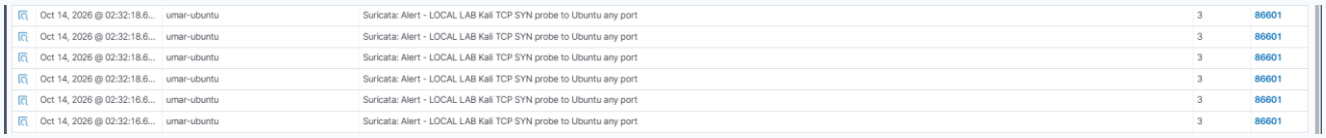
Objective. Send a small set of SYN probes and check that Suricata detects them and Wazuh receives the alerts.

Procedure. I ran Nmap from Kali against the Ubuntu host-only address, with a fixed list of ports and slow timing:

```
sudo nmap -sS -p 21,22,80,445,3306,8080 -T2 --reason 192.168.56.108
```

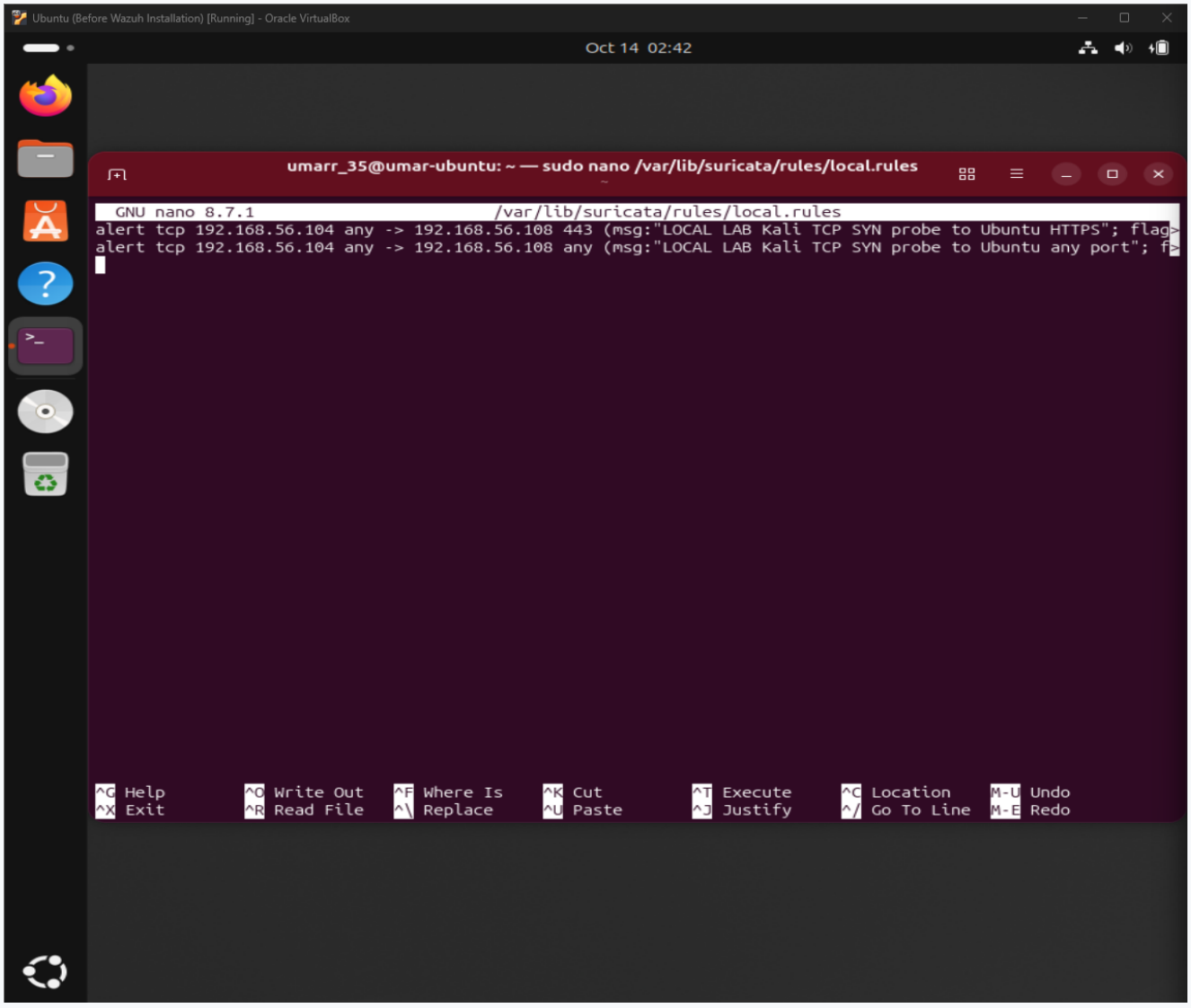
Observed result. Nmap showed the host as up. Port 22 (SSH) was open, and ports 21, 80, 445, 3306 and 8080 were closed in this scan. Suricata raised SYN-probe alerts from 192.168.56.104 to 192.168.56.108, and Wazuh showed the events through built-in rule 86601 with my custom Suricata signature ID 1000002.

Interpretation. This shows that the network detection path works for this kind of traffic. The result only covers the ports I chose and how the system was at that time.



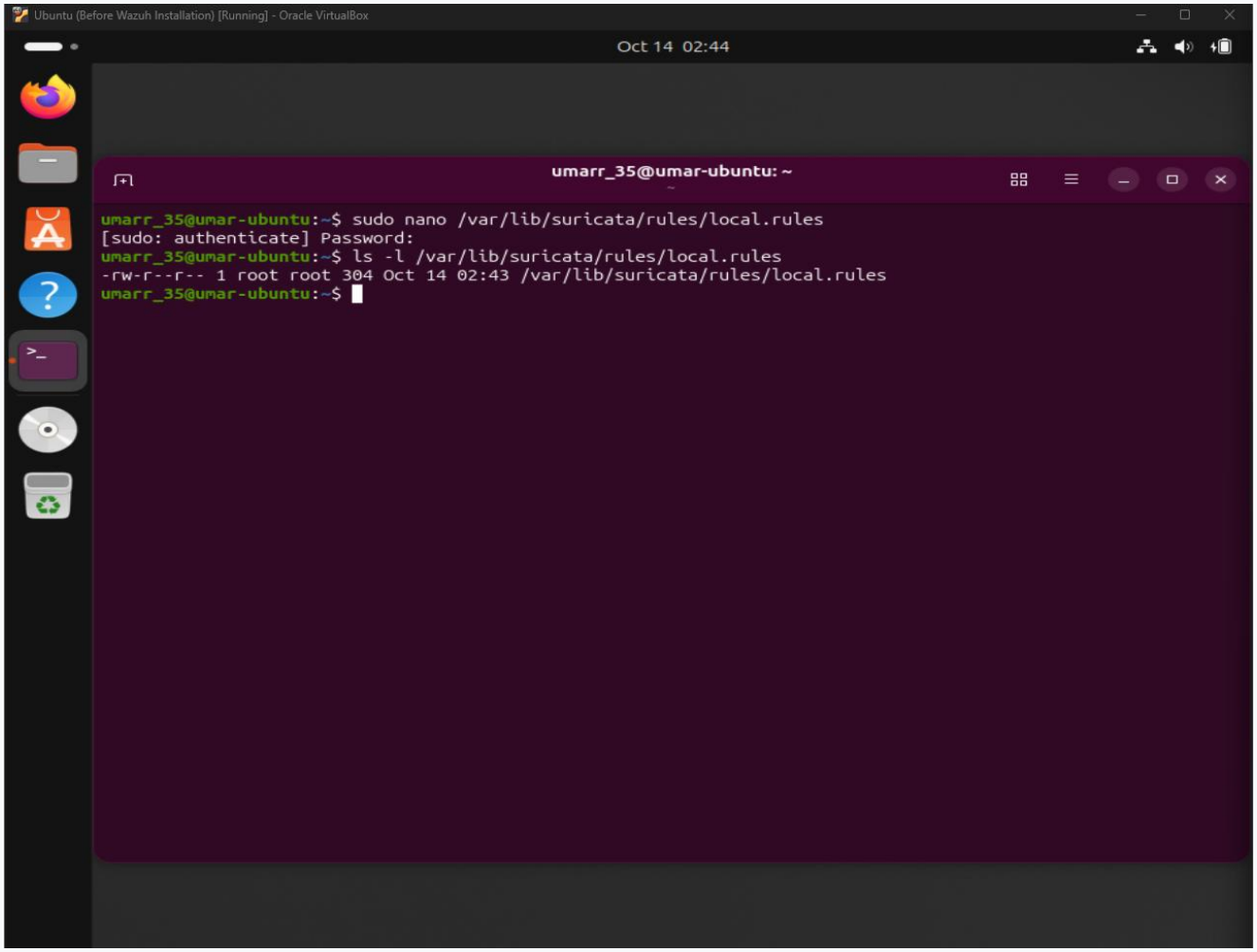
File: 13-wazuh-suricata-any-port-probe-alert.png
Wazuh events showing Suricata any-port SYN-probe alerts from the controlled scan.

Figure 10. Wazuh events showing Suricata any-port SYN-probe alerts from the controlled scan.



File: 14-suricata-custom-detection-rules.png
Suricata local.rules containing the custom HTTPS-specific and any-port signatures.

Figure 11. Suricata local.rules containing the custom HTTPS-specific and any-port signatures.



File: 15-suricata-custom-rule-file-location.png
Terminal evidence showing the location of the Suricata custom rule file.

Figure 12. Terminal evidence showing the location of the Suricata custom rule file.

7.4 Test D – Correlating repeated SYN probes

Objective. Turn many single-probe events into one higher-severity possible-port-scan alert.

Procedure. I set up custom Wazuh rule 100200 to match the Suricata alert with signature ID 1000002. Rule 100201 then looks for six matches within ten seconds from the same src_ip. I checked the configuration with wazuh-analysisd -t, restarted the manager, and ran the Nmap test again.

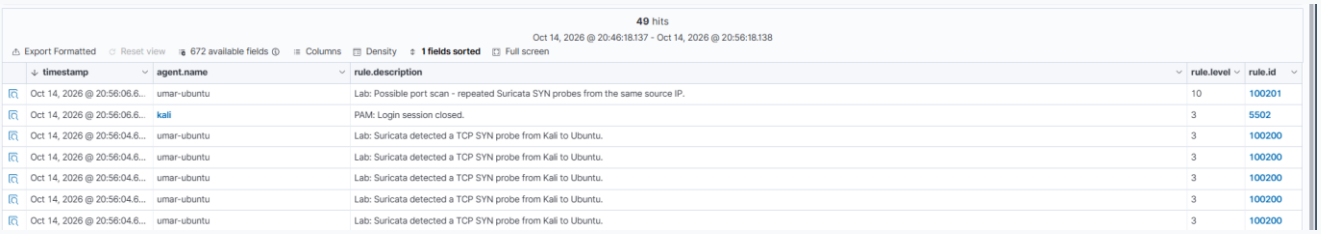
Observed result. Rule 100200 created an event for each probe. Wazuh then showed a level 10 alert for custom rule 100201 with the description “Lab: Possible port scan - repeated Suricata SYN probes from the same source IP.” It appeared after several matching probe events from the same source.

Interpretation. This is basic event correlation. The rule counts repeated matches from one source IP in a short time.



File: 17-wazuh-custom-suricata-probe-rule.png
Custom Wazuh rule 100200 configuration.

Figure 13. Custom Wazuh rule 100200 configuration.



File: 16-wazuh-port-scan-correlation-alert.png
Wazuh rule 100201 level 10 correlation alert.

Figure 14. Wazuh rule 100201 level 10 correlation alert.

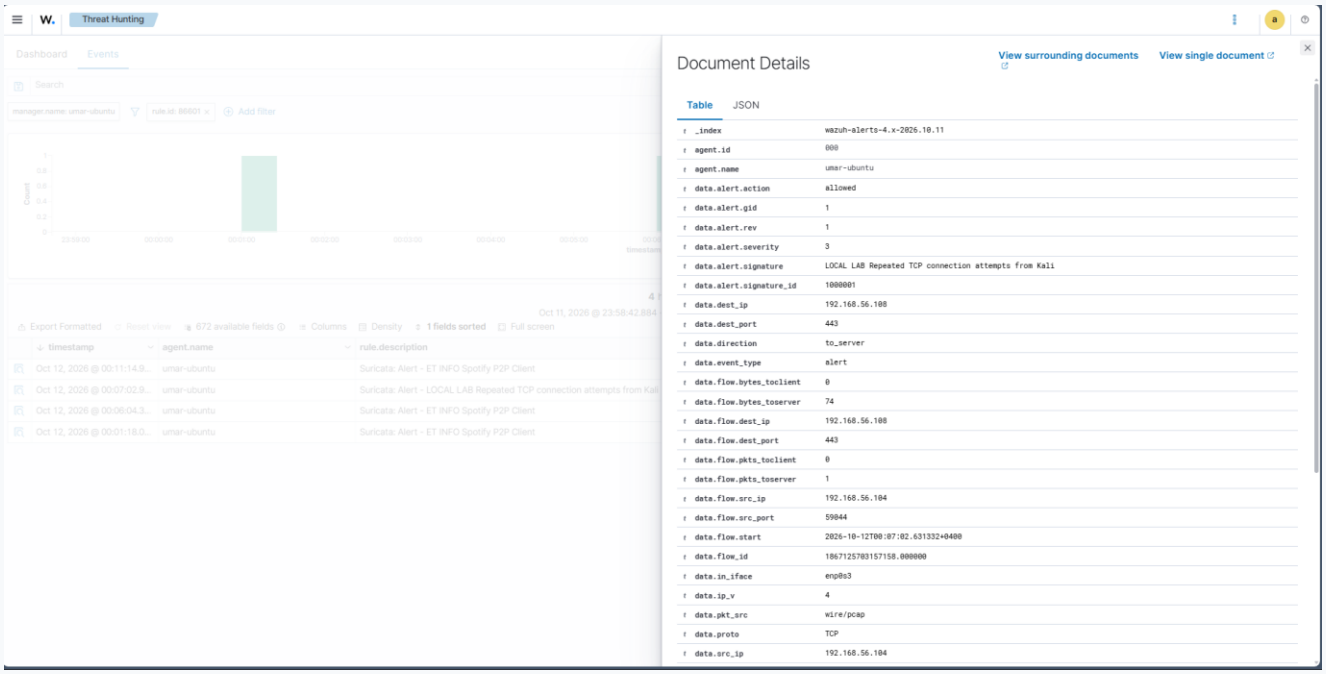


File: 18-wazuh-port-scan-alert-details.png
Expanded details for rule 100201.

Figure 15. Expanded details for rule 100201.

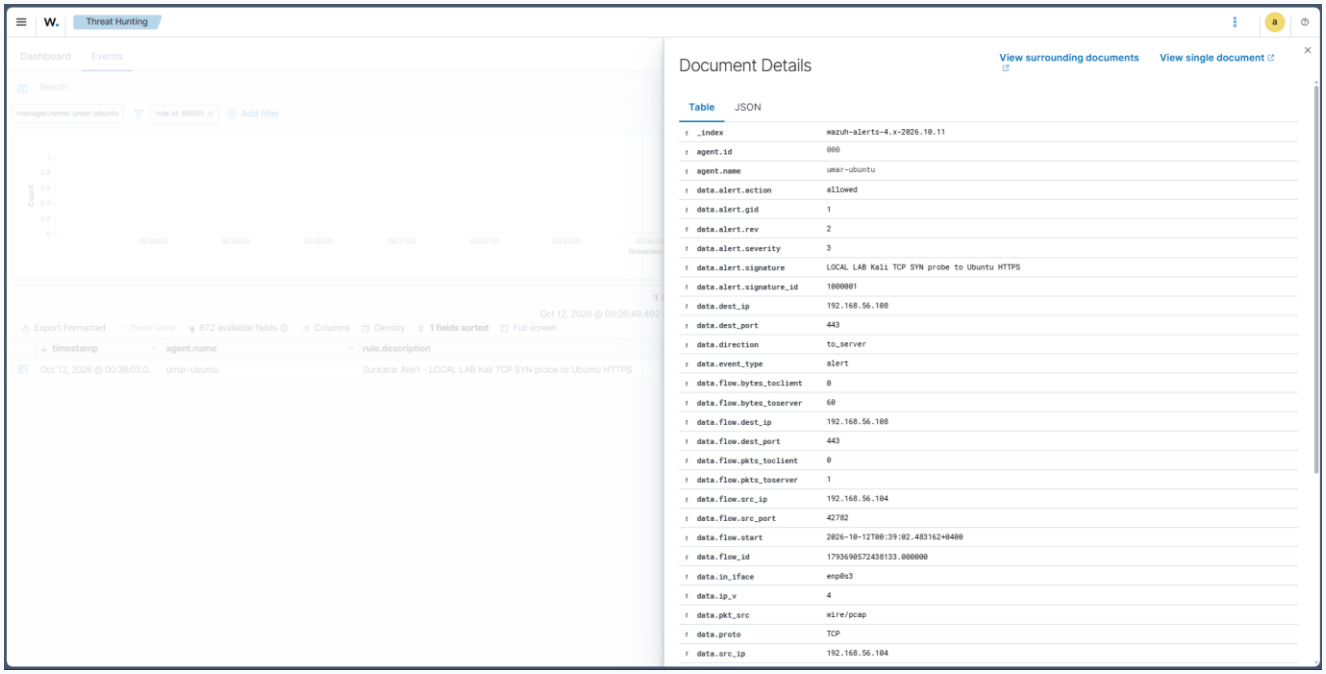
7.5 Earlier signature evidence

These two screenshots show an earlier version of my Suricata signature, which only looked for SYN probes to Ubuntu on port 443 (HTTPS). They are from before the any-port signature used in Test D.



File: 11-wazuh-suricata-custom-nmap-detection-alert.png
Earlier custom Suricata-signature alert; verify signature ID/revision.

Figure 16. Earlier custom Suricata signature alert in Wazuh.



File: 12-wazuh-suricata-nmap-syn-probe-detected.png

Figure 17. Targeted SYN probe to Ubuntu HTTPS (port 443).

8. Detection Engineering

8.1 Suricata rules

These signatures are in /var/lib/suricata/rules/local.rules. The first matches TCP SYN packets from Kali to Ubuntu port 443. The second matches TCP SYN probes from Kali to any port on Ubuntu.

```
alert tcp 192.168.56.104 any -> 192.168.56.108 443 (msg:"LOCAL LAB Kali TCP SYN probe to Ubuntu HTTPS"; flags:S;
flow:stateless; sid:1000001; rev:2;)
alert tcp 192.168.56.104 any -> 192.168.56.108 any (msg:"LOCAL LAB Kali TCP SYN probe to Ubuntu any port";
flags:S; flow:stateless; sid:1000002; rev:1;)
```

Both rules use the lab's current IP addresses. If DHCP changes either address, the rules will stop matching until I update them. My custom Wazuh rules are built on the any port rule.

8.2 Wazuh rules

These are the custom rules in /var/ossec/etc/rules/local_rules.xml:

```
<group name="local,suricata,recon,">

  <rule id="100200" level="3">
    <if_sid>86601</if_sid>
    <field name="alert.signature_id">^1000002$</field>
    <description>Lab: Suricata detected a TCP SYN probe from Kali to Ubuntu.</description>
    <group>recon,</group>
  </rule>

  <rule id="100201" level="10" frequency="6" timeframe="10">
    <if_matched_sid>100200</if_matched_sid>
    <same_field>src_ip</same_field>
```

```
    <description>Lab: Possible port scan - repeated Suricata SYN probes from the same source IP.</description>
    <group>recon,</group>
  </rule>

</group>
```

8.3 What each rule does

Suricata signature 1000002. This fires when Kali sends a TCP SYN packet to any port on the Ubuntu machine. Suricata writes each match to eve.json, and Wazuh's built-in rule 86601 picks up those alerts.

Rule 100200. This rule is built on 86601. It only matches when the Suricata signature ID is 1000002, so other Suricata alerts are ignored. It is a low-level (level 3) rule that labels each single SYN probe, so every probe becomes its own searchable event.

Rule 100201. This rule watches for 100200. If it matches six times within ten seconds from the same src_ip, it raises a level 10 alert. The same_field setting is what ties the matches to one source. So one scan from Kali gives one high level alert instead of lots of separate low level events.

9. Troubleshooting

Not everything worked the first time. This section covers three problems I ran into. I used AI tools like ChatGPT and Claude to help me work out what was going wrong and what to try next.

9.1 Nmap scan not showing up in Wazuh

Problem. I ran an Nmap scan from Kali against Ubuntu and expected Wazuh to show it, but no Nmap related events appeared. A scan working does not mean the SIEM will see it. The target has to produce the right logs, and those logs have to reach the SIEM.

What I checked. I looked in several places on Ubuntu for evidence of the scan:

- Kernel logs (journalctl -k) did not show anything useful.
- The UFW firewall was inactive, so there were no firewall logs.
- tcpdump on the host-only interface enp0s3 showed TCP SYN and RST packets from Kali (192.168.56.104) to Ubuntu (192.168.56.108), including ports like 22, 23, 443 and 445.

So the scan traffic was reaching Ubuntu, but nothing was turning it into an alert.

What fixed it. I used Suricata as a network intrusion detection tool. I wrote custom Suricata signatures for TCP SYN probes from Kali to Ubuntu, and Suricata wrote alerts to /var/log/suricata/eve.json. Then I set up Wazuh to collect that file, and the Suricata alerts showed up in the Wazuh dashboard.

Lesson. A SIEM can only detect what its data sources show it. Good detection needs the right logs, and those logs need to reach the SIEM.

9.2 Suricata and Wazuh rule files

Problem. Both Suricata and Wazuh use rules, so at first I mixed them up. They do different jobs. Suricata looks at network traffic and creates alerts. Wazuh reads incoming events and applies its own detection and correlation rules. A rule in the wrong file or the wrong format will not work.

What I worked out. There are two separate rule files:

- Suricata rules go in /var/lib/suricata/rules/local.rules. They find suspicious network traffic and create Suricata alerts.
- Wazuh rules go in /var/ossec/etc/rules/local_rules.xml. They read the Suricata alerts and create Wazuh events, including correlated alerts.

What I did. I built and tested the rules in stages instead of assuming a written rule would work. In Suricata I wrote one signature for port 443 and a wider one for any TCP port. In Wazuh, rule 100200 matched Suricata signature ID 1000002, and rule 100201 raised a level 10 alert after six matches within ten seconds from the same source IP. After that I restarted the Wazuh manager and ran the scan again.

What went wrong. Rule 100201 did not trigger at first. I fixed it by changing the source-IP condition to <same_field>src_ip</same_field>. After that, the scan triggered the level 10 alert.

Lesson. Writing a rule is not enough. I had to use the right file and format, check the field names against real event data, and test that the alert fires.

9.3 Custom test logs not showing in Wazuh

Problem. I tried to create custom WAZUH_LAB_TEST messages on Kali to check that custom events would show up in Wazuh. They did not appear in the dashboard, and I could not tell if the problem was creating the log, the agent collecting it, or the events reaching the manager.

What I checked. The rsyslog service was missing at first, so I installed and started it and set the Wazuh agent to watch /var/log/syslog. I confirmed that the messages existed locally and that the agent was set to read the file.

Result. I could not prove that the messages reached Wazuh and showed up in the dashboard, so this problem is still not fully solved. Seeing a message in a local file, and the agent watching that file, was not enough to show it worked from start to finish.

Lesson. Creating a log on a machine is different from seeing it in the SIEM. A custom event has to be collected, forwarded and processed, and a problem at any step can stop it from appearing in the dashboard.

10. Findings and Discussion

ID	Finding	Evidence	Meaning
F-01	Wazuh dashboard accessible.	Figure 3	The main page for investigating events was working.
F-02	Kali agent was Active.	Figure 4	The agent connection was working.
F-03	SSH failed login detected.	Rule 5710, level 5; Figures 6 and 7	Login events reached Wazuh.
F-04	Repeated SSH failures (Hydra test) raised a higher-severity alert.	Built-in rule 5712, level 10; Figures 8 and 9	The built-in rule works. I did not write it.
F-05	Suricata SYN-probe alerts were collected.	Rule 86601, signature ID 1000002; Figure 10	Network data was available in Wazuh.
F-06	Repeated probe events triggered correlation.	Custom rule 100201, level 10; Figures 14 and 15	A possible scan pattern was picked up in the test.

10.1 What worked

The best result was the full chain from test action to searchable logs to an alert with a clear rule behind it. The SSH tests covered login monitoring on the endpoint, and the Nmap test covered network visibility through

Suricata. The custom Wazuh rules made repeated probes much easier to spot than a long list of single low level events.

10.2 What I learned from investigating alerts

- A rule ID and level matter, but so do the fields and context of the alert.
- One failed login and many failed logins are different signals.
- Suricata shows network activity that login logs alone do not.
- A rule can be written correctly and still not work, so I have to test it.
- A SIEM can only detect what its data sources show it.

11. Limitations

11.1 Small, shared setup

Wazuh and the monitored Ubuntu machine share one VM. This is easy to set up, but if that VM fails I lose both. A real setup would keep them apart.

11.2 Limited testing

I only tested a few SSH and Nmap scenarios. The lab does not cover many kinds of attacks and does not measure overall detection coverage.

11.3 Source-based correlation

Rule 100201 counts repeated matches from one src_ip within ten seconds. It does not check for different ports, the intent behind the scan, or other scanning methods.

11.4 No false-positive testing

I only tested the custom rules with the lab scan. I did not test them against normal traffic.

12. Conclusion

This project gave me a small SIEM/SOC lab that brings together Wazuh monitoring, Suricata network data and custom correlation rules. The SSH tests showed that I could see failed logins, and that a built in rule raises a higher severity alert when failures repeat. The Nmap scan created Suricata SYN-probe events, and custom rule 100201 raised a level 10 possible-port-scan alert after repeated matches from the same source.

The biggest lesson was to check the whole detection path. Installing the tools or writing a rule was not enough. I had to create realistic events, check what Wazuh really received, fix the correlation rule and test again. The troubleshooting also showed me that some problems, like the custom log collection, can take extra digging to solve.